Basics

Variables, types, operators

Output & Variables

`console.log("Hi")`	print
`alert("Hi")`	popup
`const x = 5`	cannot reassign
`let y = 3.14`	can reassign
`var z = 1`	old style, avoid
`typeof x`	get the type

Types

`42  3.14`	number
`"text"  'text'  \`text\``	string
`true  false`	boolean
`null  undefined`	empty / not set
`123n`	BigInt
`Symbol("id")`	unique key

Operators

`+  -  *  /  %  **`	math
`===  !==  <  >`	strict compare
`&&  ||  !`	logic
`??  ?.`	nullish / optional chain
`x ||= 1   x ??= 1`	assign shortcuts
`` `Hi ${name}` ``	template string
`// one line   /* many */`	comments

Operators & Types

Compare and convert

Logic & Shortcuts

`a && b  a || b`	and / or
`!a  !!a`	not / to boolean
`a ?? b`	default if null/undefined
`a?.b  a?.[0]  f?.()`	safe access
`a ||= 1  a &&= 1  a ??= 1`	logical assign
`x ** 2`	power
`x++  x--`	add / subtract one

Type Conversion

`Number("5")  +"5"`	text to number
`String(5)  5 + ""`	number to text
`Boolean(x)`	to true / false
`parseInt("42px", 10)`	read leading int
`parseFloat("3.5x")`	read leading float
`Number.isNaN(x)`	is it NaN?

Checks

`typeof x`	type name
`Array.isArray(x)`	is an array?
`x instanceof Date`	is this class?
`Object.is(a, b)`	strict same value
`falsy: 0 "" null undefined NaN`	values that are false

Strings & Numbers

Common methods

Strings

`s.length`	string length
`s.toUpperCase()  s.toLowerCase()`	change case
`s.trim()`	trim spaces
`s.slice(1, 3)`	part of string
`s.split(",")`	string to array
`s.replace("a", "b")`	replace first
`s.replaceAll("a", "b")`	replace all
`s.includes("a")`	contains?
`s.startsWith("a")  s.endsWith("z")`	start / end
`s.indexOf("a")`	index or -1
`s.padStart(5, "0")`	pad left
`s.repeat(3)`	repeat

Numbers & Math

`Number("5")  parseInt("5")`	text to number
`n.toFixed(2)`	2 decimals
`Math.round(x)  Math.floor(x)`	round / down
`Math.ceil(x)  Math.abs(x)`	up / absolute
`Math.max(a, b)  Math.min(a, b)`	largest / smallest
`Math.random()`	0 to 1
`Math.sqrt(x)  Math.pow(a, b)`	root / power
`Number.isInteger(x)`	whole number?
`isNaN(x)`	not a number?

Arrays

Lists of values

Add / Remove

`arr.push(x)  arr.pop()`	end add / remove
`arr.unshift(x)  arr.shift()`	start add / remove
`arr.splice(i, 1)`	remove at i
`arr.slice(1, 3)`	copy part
`[...a, ...b]`	merge arrays

Loop & Transform

`arr.forEach(fn)`	run for each
`arr.map(x => x * 2)`	transform each
`arr.filter(x => x > 1)`	keep matches
`arr.reduce((a, b) => a + b, 0)`	fold to one value
`arr.find(fn)  arr.findIndex(fn)`	first match
`arr.some(fn)  arr.every(fn)`	any / all
`arr.flat()  arr.flatMap(fn)`	flatten

Search & Order

`arr.includes(x)  arr.indexOf(x)`	contains / position
`arr.sort((a, b) => a - b)`	sort numbers
`arr.reverse()`	reverse in place
`arr.join(", ")`	array to string
`Array.from({length: 3})`	make array
`Array.isArray(x)`	is array?
`const [a, b] = arr`	destructure

Destructuring & Spread

Unpack data

Destructuring

`const [a, b] = arr;`	array items
`const [x, ...rest] = arr;`	first + the rest
`const [, second] = arr;`	skip an item
`const { name, age } = o;`	object keys
`const { name: n } = o;`	rename on unpack
`const { a = 5 } = o;`	default value
`[a, b] = [b, a];`	swap values

Spread & Rest

`[...a, ...b]`	join arrays
`{ ...o, x: 1 }`	copy and change
`function f(...args) { }`	collect arguments
`Math.max(...arr)`	spread as arguments

In Parameters

`function f({ a, b }) { }`	unpack an object arg
`function f([a, b]) { }`	unpack an array arg
`function f(a = 1) { }`	default argument
`const { a: { b } } = o;`	nested unpack
`for (const [k, v] of Object.entries(o))`	loop pairs

Objects & JSON

Key and value

Objects

`const o = {name: "A", age: 5}`	create object
`o.name  o["name"]`	read property
`o.age = 6`	set property
`delete o.age`	remove property
`"name" in o`	key exists?
`const {name, age} = o`	destructure
`{...o, age: 7}`	copy + change

Object Methods

`Object.keys(o)`	all keys
`Object.values(o)`	all values
`Object.entries(o)`	key-value pairs
`Object.assign({}, o)`	shallow copy
`Object.freeze(o)`	make read-only
`structuredClone(o)`	deep copy

JSON

`JSON.stringify(o)`	object to text
`JSON.parse(s)`	text to object
`JSON.stringify(o, null, 2)`	pretty print

Classes & this

OOP in JavaScript

Class Basics

`class A { }`	define a class
`constructor(x) { this.x = x; }`	setup code
`method() { return this.x; }`	instance method
`static create() { }`	class-level method
`new A(5)`	make an instance
`#secret = 1;`	private field

Inheritance & Accessors

`class B extends A { }`	inherit
`super(x);`	call parent constructor
`super.method();`	call parent method
`get full() { }`	computed property
`set full(v) { }`	setter
`static #count = 0;`	private static

this & Binding

`fn.call(obj, a)`	call with this
`fn.apply(obj, [a])`	call with array args
`fn.bind(obj)`	fixed this copy
`() => this.x`	arrow keeps outer this
`Object.create(proto)`	object from prototype

Functions & Flow

Define, loop, decide

Functions

`function f(a, b) { return a + b }`	declaration
`const f = (a, b) => a + b`	arrow function
`function f(a = 1, ...rest)`	default / rest
`f(...arr)`	spread args
`(() => { })()`	run instantly

Conditions

`if (x) { } else if (y) { } else { }`	branches
`a ? b : c`	ternary
`switch (x) { case 1: break; }`	many cases

Loops

`for (let i = 0; i < 5; i++)`	classic loop
`for (const x of arr)`	loop values
`for (const k in obj)`	loop keys
`while (cond) { }`	loop while true
`break  continue`	exit / skip

Errors

`try { } catch (e) { } finally { }`	handle errors
`throw new Error("msg")`	throw error

Map, Set & Symbols

Modern collections

Map

`const m = new Map();`	key-value store
`m.set("a", 1)`	add / update
`m.get("a")  m.has("a")`	read / exists
`m.delete("a")  m.clear()`	remove
`m.size`	number of items
`for (const [k, v] of m) { }`	loop pairs

Set

`const s = new Set([1, 2, 2]);`	unique values
`s.add(3)  s.has(3)`	add / exists
`s.delete(3)`	remove
`[...s]`	set to array
`new Set([...a].filter(x => b.has(x)))`	intersection

Other

`new WeakMap()  new WeakSet()`	do not block cleanup
`Symbol("id")`	unique key
`Symbol.iterator`	make iterable
`function* gen() { yield 1; }`	generator function
`structuredClone(o)`	deep copy

DOM & Events

Control the page

Select

`document.getElementById("id")`	by id
`document.querySelector(".a")`	first match
`document.querySelectorAll("p")`	all matches
`el.closest(".box")`	nearest parent

Change

`el.textContent = "Hi"`	set text
`el.innerHTML = "<b>Hi</b>"`	set HTML
`el.style.color = "red"`	inline style
`el.classList.add("on")`	add class (remove, toggle)
`el.setAttribute("href", "#")`	set attribute
`document.createElement("div")`	make element
`parent.append(el)  el.remove()`	add / remove

Events

`el.addEventListener("click", fn)`	listen
`e.preventDefault()`	stop default
`e.target`	clicked element
`"click" "input" "submit" "keydown"`	common events
`DOMContentLoaded`	page ready

Async & Modules

Promises and imports

Async

`setTimeout(fn, 1000)`	run later
`setInterval(fn, 1000)`	repeat
`new Promise((res, rej) => { })`	make promise
`p.then(fn).catch(fn)`	handle result
`async function f() { }`	async function
`await p`	wait for promise
`Promise.all([p1, p2])`	wait for all

Fetch

`const r = await fetch(url)`	request
`await r.json()`	read JSON body
`fetch(url, {method: "POST", body})`	send data

Storage

`localStorage.setItem("k", "v")`	save
`localStorage.getItem("k")`	read

Modules

`export const a = 1`	named export
`export default fn`	default export
`import fn, { a } from "./f.js"`	import

Dates & Intl

Time and formatting

Date

`new Date()`	now
`new Date("2026-05-20")`	from text
`Date.now()`	milliseconds now
`d.getFullYear()  d.getMonth()`	year / month (0-11)
`d.getDate()  d.getDay()`	day / weekday
`d.setDate(d.getDate() + 7)`	add 7 days

Format

`d.toISOString()`	standard text
`d.toLocaleDateString()`	local date
`d.toLocaleTimeString()`	local time
`d.toLocaleDateString("en", {month:"long"})`	month name

Intl

`new Intl.NumberFormat("en-US")`	number format
`nf.format(1234.5)`	1,234.5
`new Intl.DateTimeFormat("en")`	date format
`new Intl.RelativeTimeFormat("en")`	"3 days ago"
`(1234.5).toLocaleString()`	quick local format
`n.toFixed(2)`	2 decimals as text

Regex

Pattern matching

Create & Test

`/ab+c/i`	regex literal
`new RegExp("a+", "g")`	build from text
`re.test(s)`	true / false
`s.match(re)`	matches
`s.matchAll(re)`	all matches with groups
`s.replace(re, "x")`	replace

Symbols

`\d  \w  \s`	digit / word / space
`\D  \W  \S`	the opposites
`^  $`	start / end
`.  [abc]  [^abc]`	any / set / not set
`a*  a+  a?`	0+ / 1+ / optional
`a{2,4}`	2 to 4 times

Groups & Flags

`(abc)`	capture group
`(?<year>\d{4})`	named group
`(?:abc)`	group, no capture
`a|b`	a or b
`g  i  m  s`	global / case / lines / dotall
`\b`	word boundary

Browser APIs

Storage and more

Storage

`localStorage.setItem("k", v)`	save text
`localStorage.getItem("k")`	read text
`localStorage.removeItem("k")`	delete one
`JSON.stringify(obj)`	object to text
`JSON.parse(text)`	text to object
`sessionStorage`	cleared with tab

Page & Window

`location.href`	current address
`history.pushState({}, "", "/a")`	change address
`window.innerWidth`	viewport width
`scrollTo({ top: 0, behavior: "smooth" })`	scroll to top
`navigator.clipboard.writeText(s)`	copy text

Timers & Observers

`setTimeout(fn, 1000)`	run once later
`setInterval(fn, 1000)`	repeat
`clearInterval(id)`	stop repeating
`requestAnimationFrame(fn)`	next frame
`new IntersectionObserver(fn)`	watch visibility
`navigator.geolocation`	user location

Tips & Tricks

Handy one-liners

Handy

`[...new Set(arr)]`	remove duplicates
`arr.at(-1)`	last item
`Object.fromEntries(entries)`	entries to object
`console.table(arr)`	table in console
`console.error("x")`	error log

Debug

`debugger;`	pause in devtools
`console.time("t")`	time code
`"use strict";`	stricter rules

Good Habits

`Prefer const, then let`	avoid var
`Use === not ==`	safe compare
`Format with Prettier`	clean code

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily